Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Rootrix Virtual File system

Team Name: *Team #*	*Rootrix*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Naithani, Tanish – 25100011477* *25100011477@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Singh, Tushar – 2510011704* *2510011704@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Tayal, Rounak – 2510370114* *2510370114@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Rawal, Aditya – 2510380204* *2510380204@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

(Brief restatement of your project's main goal. Max 300 words).

Rootrix VFS is a Virtual File System that we built in C++ for our Data Structures project. Our main goal is to understand what really happens inside an operating system when we create a folder, save a file or delete something, by building a small file system of our own from scratch.

Instead of letting Windows manage our data, our program creates its own virtual disk in memory. The disk is divided into fixed-size blocks, and a bitmap tells us which blocks are free and which are in use. Every file and folder is stored as an inode, which keeps its name, type, parent folder, size, permissions and the list of blocks where its data is saved. The user works with the system through 19 terminal-like commands, such as mkdir, cd, write, cat, cp, mv, rm, chmod, lock and find.

With this project we are using arrays, trees, bitmaps and hash tables on a real problem instead of only reading about them. Phase 2 is now complete with read-only files, password-locked folders, fast search with a hash index and data that stays saved after closing. In Phase 3 we will add an LRU cache and journaling for crash recovery.

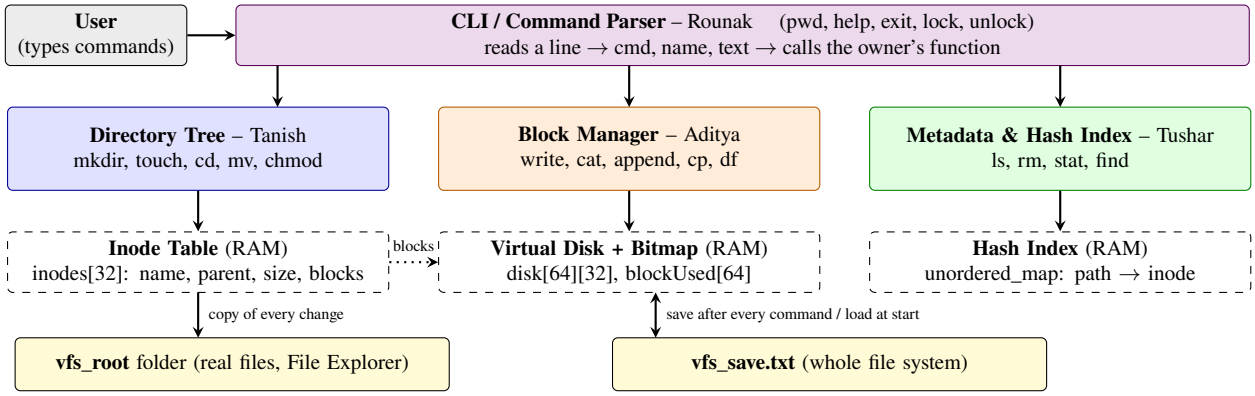
Updated Project Approach and Architecture

(Describe your current approach, including system design, communication protocols, libraries used, etc. Max 300 words).

We are using plain C++ (C++11) with the g++ compiler (MinGW on Windows). No external library is used, only the C++ standard library (iostream, string, vector, sstream, fstream, unordered_map), plus direct.h for real folders on Windows.

The code is split into four modules, one per member, as shown in the diagram. The files are linked in a chain with #include, so we compile only one main file.

Each inode stores the number of its parent folder, so we need no pointers: a folder's contents are all inodes with that parent, and mv only changes the name or parent number. Character i of a file goes to block $i/32$ at position $i \bmod 32$. The hash index maps every full path to its inode number. Names ignore upper and lower case, like Windows. Every command checks for problems first and only then changes anything.



Tasks Completed

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Inode table, path building, case-insensitive names; **mkdir**, **touch**, **cd**, **mv**, **chmod**	Tanish Naithani (Team Lead)
Virtual disk and bitmap, saving data after restart; **write**, **cat**, **append**, **cp**, **df**	Aditya Rawal
Hash index; **ls**, **rm**, **stat**, **find**; test program with 16 tests and 39 checks	Tushar Singh
Command-line interface, real folder copy (vfs_root); **pwd**, **help**, **exit**, **lock**, **unlock**	Rounak Tayal
Work division by roles and joining all modules into one program	All members

Challenges/Roadblocks

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

Old compiler: g++ on one laptop had no filesystem header, so we rewrote that part with older functions like _mkdir and remove.

Windows blocking our exe: Smart App Control blocked the unsigned program, so we run it on a laptop where this feature is off.

Data lost on restart: everything was only in RAM, so we now save the file system to a text file after every command and load it at startup.

Upper/lower case: our mentor made “Rahul” and “rahul”. Our VFS kept both, but Windows sees one name, so only one real folder appeared. We fixed it by comparing names without case, like Windows.

Small disk: the disk holds only 2048 characters, so long text is safely refused with “disk full”. Changing two constants makes it bigger, and very long input is given from a file.

Tasks Pending

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
LRU cache for recently used blocks	Aditya Rawal
Bigger, growing disk; Phase 3 integration	Tanish Naithani
Journaling for crash recovery and its tests	Tushar Singh
crash and recover commands; README and user guide	Rounak Tayal
Final testing on every team laptop	All members

Project Outcome/Deliverables

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The main outcome is a working command-line Virtual File System in C++ with 19 commands to create, read, edit, copy, move, search and delete files and folders. The code is divided into clear modules, so each of us can explain our own part.

Along with it we have a test program that checks real values, storage that keeps data after the program is closed, read-only files, password-locked folders and quick search by full path using a hash index. In the final phase we will add an LRU cache and journaling for crash recovery.

We will also submit documentation on how the system works and how to run it, and give a live demo.

Progress Overview

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

Phase 2 is complete. All 19 planned commands are working: mkdir, touch, cd, mv, chmod, write, cat, append, cp, df, ls, rm, stat, find, pwd, lock, unlock, help and exit. The inode table, virtual disk, bitmap, hash index and command line are finished and tested.

Ahead of schedule: saving data between runs and showing files in a real Windows folder were not planned for this stage, but both are done. We also fixed the upper/lower case issue found in review.

Behind schedule: the README is not finished yet, and we still have to test the program on every team laptop because Windows blocked it on one of them.

Next we will start Phase 3: the LRU cache and journaling.

Codebase Information

(Repository link, branch, and information about important commits.)

Repository: https://github.com/adityarawal1985-bit/virtual-file-system **Branch:** main **Language:** C++11 (g++ / MinGW)

Files: phase2_tanish_directory_tree.h, phase2_aditya_block_manager.h, phase2_tushar_metadata.h, phase2_tushar_tests.cpp, phase2_rounak_cli.h, phase2_rounak_real_folder.h, phase2_main_integration.cpp

Important commits: [add commit IDs] (1) first version with inode array and mkdir, touch, ls; (2) virtual disk and bitmap with write and cat; (3) code split into role-wise files with 10 commands; (4) real folder copy made to work on old compilers; (5) save and load after restart; (6) remaining 9 commands with hash index, security and case-insensitive names.

Testing and Validation Status

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Unit tests (test program)	Pass	16 tests, 39 checks
Manual testing of all 19 commands	Pass	Right and wrong inputs
Security (chmod, lock, unlock)	Pass	Blocked without rights
Hash index search (find)	Pass	Updates after mv
Disk full (text bigger than disk)	Pass	Refused, old data safe
Restart (save and load)	Pass	Files, locks restored
Upper/lower case names	Pass (fixed)	Rahul = rahul

Deliverables Progress

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Completed: core file system (inode table, virtual disk, bitmap, folder tree); all 19 Phase 2 commands; read-only files and password-locked folders; hash index search; saving data between runs; real folder copy (vfs_root); test program with 39 checks.

In progress: documentation and README.

Pending: LRU cache and journaling for crash recovery (Phase 3).